

CSE 312 – Operating Systems · Spring 2026

FILE SYSTEMS

Kapsamli Konu Anlatimi ve Soru Cozumleri

Prof. Dr. Yakup Gene tarafından islenen konular

- Kapsam:** File sistemi tasarimi, dosya tipleri, dizin yapilari, disk alani yonetimi, i-node, FAT, journaling, cache, backup, fsck, VFS, UNIX & Windows FS
- Bolumler:** 4.1 – 4.7 (Tanenbaum Bolum 4)
- Sorular:** 50 kitap soru cozu, sinav sorulari, final tekrari

Contents

1	Giris: File System Nedir ve Neden Gereklidir?	5
1.1	Temel Motivasyon	5
1.2	Tasarim Kararlari (Design Decisions)	5
1.2.1	Dosya Adlandirma (File Naming)	6
1.2.2	Diger Disk Suruculerin Adlandirilmesi	6
2	Dosya Tipleri ve Yapilari (File Types & Structures)	6
2.1	Dosya Tipleri	6
2.2	Dosya Yapilari	7
3	Metadata: Dosya Hakkinda Veriler	7
3.1	Metadata Nedir?	7
3.1.1	Linux <code>struct stat</code> – Gercek Bir Ornek	8
3.2	Metadata Nerede Saklanır?	8
4	Dosya Operasyonlari (File Operations)	9
4.1	Temel Sistem Cagrisi Seti	9
4.2	Open() Sistem Cagrisi – Neden Gereklidir?	9
4.3	Unix'te Dosya Okuma Surecinin Detayi	9
4.4	Unix'te Dosya Yazma Surecinin Detayi	10
4.5	Seek	10
4.6	Dosya Acma ve Kapama Mekanizmasi – Detayli Sekil	10
5	Dizinler (Directories)	10
5.1	Dizin Yapilari	10
5.1.1	Tek Seviyeli Dizin	10
5.1.2	Hiyerarsik Dizin	10
5.2	Mutlak ve Goreli Yollar	11
5.3	Dizin Operasyonlari	11
5.4	Dizin Uygulamasi (Directory Implementation)	11
5.4.1	Uzun Dosya Adlari	12
6	Disk Alan Yonetimi – Blok Tahsisi (Disk Space Management)	12
6.1	File System Genel Yerlesimi	12
6.2	Blok Boyutu Secimi	12
6.3	Allocation Yontemleri – Karsilastirmali Inceleme	13
6.3.1	1. Contiguous Allocation (Surekli Tahsis)	13

6.3.2	2. Linked List Allocation (Bagli Liste Tahsisi)	14
6.3.3	3. File Allocation Table (FAT)	14
6.3.4	4. Indexed Allocation – i-node	15
6.3.5	i-node ile Maksimum Dosya Boyutu	15
6.3.6	Allocation Yontemlerinin Karsilastirmasi	17
7	Bos Alan Yonetimi (Free Space Management)	17
7.1	Free List ve Bitmap Karsilastirmasi	17
7.1.1	Linked Free List	17
7.1.2	Bitmap	17
8	Paylasilan Dosyalar ve Linkler (Shared Files & Links)	18
8.1	Hard Link (Sabit Baglanti)	18
8.2	Symbolic Link (Sembolik Baglanti)	19
9	Journaling File Systems	19
9.1	Cokme Sonrasi Tutarlilik Sorunu	19
9.2	Journaling (Log-Structured / Journaling File System)	20
10	Virtual File System (VFS)	20
10.1	VFS Nedir ve Neden Gereklidir?	20
11	File System Tutarliliği ve Tamiri (Consistency & fsck)	21
11.1	Blok Tutarliliği	21
12	File System Performansi	22
12.1	1. Caching (Tampon Onbellekleme)	22
12.2	2. Block Read-Ahead (Blok Onokuma)	23
12.3	3. Disk Kolu Hareketini Azaltma	23
13	File System Yedeklemesi (Backup)	23
13.1	Yedekleme Stratejileri	23
13.1.1	Fiziksel Dump (Physical Dump)	23
13.1.2	Mantiksal Dump (Logical Dump)	24
13.1.3	Mantiksal Dump Algoritmasi (4 Faz)	24
13.2	Kurtarma Algoritmasi	24
14	Dosya Sistemi Ornekleri	25
14.1	ISO 9660 – CD-ROM Dosya Sistemi	25
14.1.1	Rock Ridge Uzantilari	25
14.1.2	Joliet Uzantilari	25

14.2 MS-DOS / FAT Dosya Sistemi	25
14.2.1 Uzun Ad Destegi (Windows 98+)	26
14.3 UNIX V7 Dosya Sistemi	26
14.3.1 Dosya Bulma: /usr/ast/mbox	27
15 Unix Dosya Sistemi Islemleri – Derinlemesine Inceleme	27
15.1 . ve .. Dizin Girisileri	27
15.2 Dosya Olusturma (Unix)	27
15.3 Dosya Silme (Unix) – Unlink Mekanizmasi	28
15.4 Dizin Olusturma ve Silme	28
16 i-Node – Kapsamli Inceleme	28
16.1 i-node Nedir?	28
16.2 i-node’da Ne Var?	29
16.3 i-node ile Buyuk Dosya Destegi	29
17 Disk Kotasi (Disk Quota)	30
18 Kapsamli Soru Cozumleri	30
18.1 Teorik ve Kavramsal Sorular	30
19 Kavram Sozlugu	33
20 Sinav Cevap Kaliplari	34
21 Final Tekrari – 30 Kritik Madde	34

1 Giriş: File System Nedir ve Neden Gereklidir?

1.1 Temel Motivasyon

Bir bilgisayar, verilerini RAM’de tutabilir; ancak RAM kalıcı (persistent) değildir – güç kesildiğinde her şey kaybolur. Uzun vadeli veri saklamak için farklı bir soyutlama gerekir. **File system** (dosya sistemi), işletim sisteminin depolama kaynakları için bulunduğu soyutlama katmanıdır.

Temel Tanımlar

File (dosya): İşletim sisteminin disk veya başka kalıcı bir ortam üzerindeki veriyi organize etmek için kullandığı mantıksal depolama birimidir. Bir dosya, byte’ların sıralı bir dizisidir (Unix perspektifinden); içeriğe bakışımız ise hangi OS’te olduğumuzla değişir.

File System: Dosya adlarından fiziksel ortamdaki bir bölgeye olan eşlemedir (mapping). Aynı zamanda OS’in depolama kaynakları için sunduğu soyutlama katmanıdır.

Directory (dizin): Mantıksal olarak bir grup dosyayı barındıran "konteyner"dir. Kendi içinde başka dizinler de barındırabilen özel bir veri yapısıdır.

Uzun vadeli bilgi depolamak için üç temel gereksinim vardır:

1. Çok büyük miktarda bilgi depolanabilmeli (Petabyte’larla ölçülen miktarlar)
2. Bilgi, o veriyi kullanan süreci sonlandıktan sonra da hayatta kalmalı (**non-volatile/persistent**)
3. Birden fazla süreç aynı anda bu bilgiye erişmeli

IEEE Spectrum (Mart 2014) verilerine göre: Tek bir dijital uzun metraj film 2 petabyte veri üretir (≈ 500 bin DVD). Her yıl 700 yeni film çıkmaktadır. 1 petabyte’ın 1 gigabit ethernet üzerinden aktarımı 90 gün sürmektedir. Bu ölçek, file system tasarımının ne kadar kritik olduğunu göstermektedir.

Diski basit bir “blok dizisi” olarak düşünün. Hemen şu sorular ortaya çıkar:

- Bilgiye nasıl ulaşılacak? (indexing, naming)
- Bir kullanıcının verisi diğer kullanıcıdan nasıl korunacak? (access control)
- Hangi bloklar boş? (free space management)

Bu üç soru, file system tasarımının özünü oluşturur.

1.2 Tasarım Kararları (Design Decisions)

File system tasarımı birçok kritik kararı kapsar. Bunları sistematik şekilde inceleyelim:

1.2.1 Dosya Adlandirma (File Naming)

Konu	Aciklama ve Ornekler
Case sensitivity	Unix: buyuk/kucuk harf duyarli (<code>File.txt</code> $\neq$ <code>file.txt</code>). Windows (FAT/NTFS): tarihsel olarak duyarsiz.
Extension	Unix cekirdegi uzantiyi bilmez; uygulama katmani yorumlar. Eski DOS: “8.3” format (8 karakter ad, 3 karakter uzanti).
Karakter seti	ASCII mi? Unicode mi? Ulke bazli tanimlayici mi?
Max uzunluk	FAT-16: 8+3 karakter. NTFS, ext4: 255 karakter.
Path separatoru	Unix: <code>/</code> , Windows: <code>\</code> , Multics: <code>></code>

Farkli sistemlerde yol adlarına örnekler:

- `/this/is/a/unix.file.name` – Unix/Linux
- `q:\Windows likes\mOn0caSE` – Windows
- `>Multics>uses<for>up-a-level` – Multics
- `vms-host::device[direct.ory.list]name.type;1` – VMS (versiyon numarasına dikkat!)

1.2.2 Diger Disk Suruculerin Adlandirilmesi

Sistem	Yaklasim
Unix/Linux	Baska diskler, <code>mount</code> komutuyla dosya agacinin bir alt agaci olur. Tek bir kok agac (<code>/</code>) vardır.
Windows/VMS	Suruculer acik sure harfleriyle tanimlanir: <code>C:</code> , <code>D:</code> , vb.
Multics	Tek agac, ancak dizinler her zaman kalici olarak baglanmis disklerde.
OS/360	Acik disk tanimlayicilari ya da “system catalog” (hiyerarsik yapı, dizinler farkli disklerde olabilir).

2 Dosya Tipleri ve Yapilari (File Types & Structures)

2.1 Dosya Tipleri

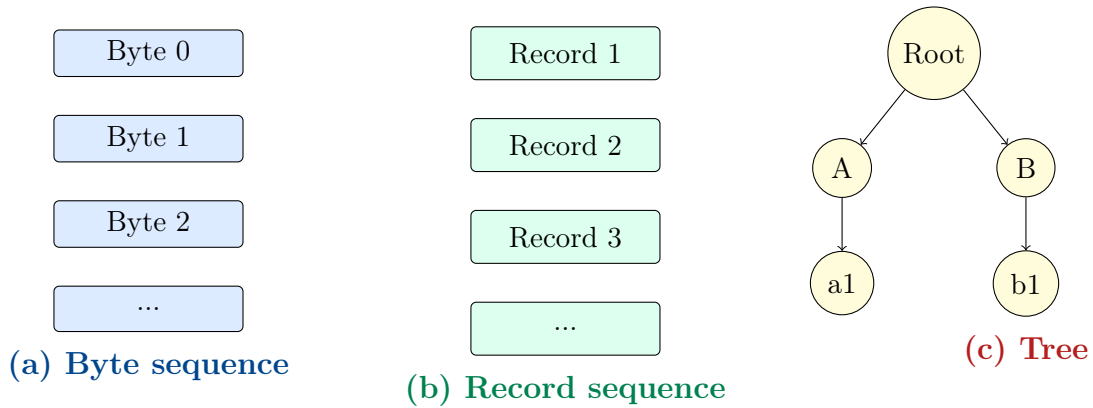
Bazi OS’lar bazi dosyalari ozel sayar:

- Unix’te cogu I/O aygiti bir dosya adiyla temsil edilir (`/dev/sda`, `/dev/null`, vb.)
- Dizinler bazen sadece dosyadir (Unix’te oyle)
- Bazi iletisim kanallari (pipe, socket) da dosya adiyla temsil edilir (named pipe, socket file)
- **Magic number:** Bazi sistemler, dosyanin basindaki birden fazla bayta bakarak tipini anlar (executable’lar icin `0x7f ELF`, JPEG icin `0xFF D8`, vb.)

“Unix’te executable dosyalar neden rastgele degerli bir magic number yerine cok ozel bir magic number kullaniyor?” – Cevap: Executable format yanlis tespit edilirse, cekirdek yanlis kodu calistirabilir. Bu hem guvenlik hem de istikrar acisindan felaket olur. Bu yuzden 0x7f ‘ELF’ gibi ozel bir imza secilmistir ki yanlislikla baska turde bir dosya executable olarak tanimlanmasin. Diger dosya turleri icin boyle bir kritik risk yoktur; yanlis tanimlanma yalnizca uygulamanin hata vermesine yol acar.

2.2 Dosya Yapilari

Dosyalar uc farkli sekilde yapilandirilabilir:



Yapi	Aciklama	Kullanim Yeri
Byte sequence	Dosya sadece byte dizisidir; anlam uygulamaya birakir	Unix, Linux, Windows
Record sequence	Dosya sabit boyutlu record’lardan olusur	Eski ana bilgisayar sistemleri
Tree	Anahtar degerli arama agaci yapisi	Bazi veritabani dosya sistemleri

ONEMLI NOKTA

Modern isletim sistemlerinin buyuk cogunlugu (Unix, Linux, Windows) dosyalari **byte sequence** olarak gorur. Dosyanin icindeki anlami anlamak isletim sisteminin degil, uygulamanin gorevidir.

3 Metadata: Dosya Hakkinda Veriler

3.1 Metadata Nedir?

Metadata, bir dosya hakkindaki veridir. “Veri hakkında veri” da denir. Metadata olmadan OS, diski sadece anlamsiz bir byte yigini olarak gorur.

3.1.1 Linux struct stat – Gercek Bir Ornek

Listing 1: Linux'ta bir dosyanin metadata yapisi

```

1 struct stat {
2     dev_t    st_dev;        /* Hangi aygit (device) uzerinde */
3     ino_t    st_ino;        /* i-node numarası */
4     mode_t   st_mode;       /* Izinler (rwxrwxrwx) + dosya tipi */
5     nlink_t  st_nlink;      /* Hard link sayisi */
6     uid_t    st_uid;        /* Sahibin kullanıcı ID'si */
7     gid_t    st_gid;        /* Sahibin grup ID'si */
8     dev_t    st_rdev;       /* Aygit dosyasi ise, major/minor numarası */
9     off_t    st_size;       /* Toplam boyut (byte) */
10    blksize_t st_blksize;    /* Tercih edilen I/O blok boyutu */
11    blkcnt_t  st_blocks;     /* Tahsis edilen 512-byte blok sayisi */
12    time_t    st_atime;      /* Son erisim zamanı */
13    time_t    st_mtime;      /* Son degistirilme zamanı */
14    time_t    st_ctime;      /* Son durum degisiklik zamanı */
15 };

```

atime (access time): Dosya okunduğunda guncellenir.

mtime (modification time): Dosyanin *icerigi* degistiginde guncellenir.

ctime (change time): Dosyanin *metadata'si* (izin, hard link sayisi, sahip) degistiginde guncellenir. NOT: ctime “creation time” (olusturma zamanı) DEGILDIR! – bu klasik bir yanlış!

3.2 Metadata Nerede Saklanır?

Bu soru, file system tasariminin en kritik kararlarından biridir. Uc secenek vardır:

Yer	Avantajlar	Dezavantajlar
Dosyanin icinde	Kolay tasınabilirlik (teorik)	Tasiyinca metadata kaybı; standart format yok (Apple'in “resource fork” yaklasimiyla yapilmistir)
Directory entry'de	Metadata'ya hizli erisim	Hard link'i zorlar (metadata kopyalari senkronize tutulmalı); izin buyur
Baska yerde (i-node)	Hard link'te kolaylık; metadata tek yerde	I/O siralamasi onem kazanir; i-node'a ekstra erisim gerekir

ONEMLI NOKTA – Unix Kararı

Unix, metadata'yi **i-node** adli ayrı bir yapıda saklar. Directory entry yalnızca dosyanin *adini* ve *i-node numarasini* tutar. Bu tasarım, hard link'leri dogal olarak destekler: iki

farkli dizin girişi aynı i-node numarasına işaret edebilir.

Ad $\xrightarrow{\text{directory}}$ i-node numarası $\xrightarrow{\text{i-list}}$ i-node $\xrightarrow{\text{blok adresleri}}$ veri bloklar

4 Dosya Operasyonlari (File Operations)

4.1 Temel Sistem Cagrisi Seti

File system ile ilgili en yaygın sistem çağrılarları:

- **Create** – Dosya oluşturun
- **Delete** – Dosyayı sil
- **Open** – Dosyayı aç (i-node belleğe al)
- **Close** – Dosyayı kapat
- **Read** – Oku
- **Write** – Yaz
- **Append** – Sona ekle
- **Seek** – Konumu değiştir
- **Get Attributes** – Özellikleri al
- **Set Attributes** – Özellikleri değiştir
- **Rename** – Yeniden adlandır

4.2 Open() Sistem Cagrisi – Neden Gereklidir?

“Unix’te `open()` sistem çağrısı mutlak olarak gerekli midir?” – **Cevap:** Mantıksal olarak gerekli değildir: `read()` ve `write()` çağrısı her seferinde dosya adını alıp, dizin aramasını yapıp, erişme izinlerini kontrol edebilir. Ancak bu **son derece verimsiz** olur. `open()` şu işi yapar:

- Dosya adını kullanarak i-node’u bulur ve belleğe alır (i-node cache)
- İzinleri bir kez kontrol eder
- Dosya tablosunda (file table) bir giriş oluşturur ve kullanıcıya file descriptor döner

Sonraki tüm `read/write` işlemleri bu file descriptor üzerinden **direk** i-node’a erişir; isim aramasına gerek kalmaz. Performans kazanımı büyüktür.

4.3 Unix’te Dosya Okuma Surecinin Detayi

1. Mevcut byte offset’i blok numarasına çevir
2. O bloku diskten oku (veya buffer cache’den al)
3. Kullanıcıya doğru byte’leri dön
4. Mevcut byte offset’i güncelle

5. Opsiyonel: Erisim sirasal gorunuyorsa, bir sonraki blok icin **read-ahead** baslatL

4.4 Unix'te Dosya Yazma Surecinin Detayi

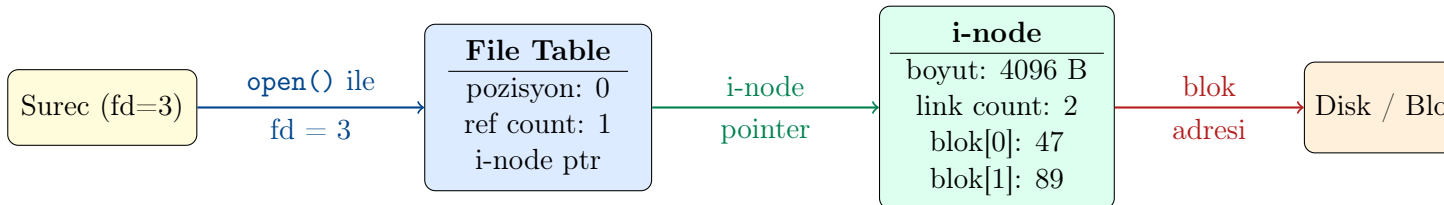
1. Bir blogun ortasina mi yaziyoruz? – Evet ise: o bloku once oku (read-modify-write)
2. Hayir ise: Freelist'ten yeni blok al, i-node'a ekle
3. Veriyi buffer'a kopyala
4. Buffer'i **dirty** olarak isaretle (zamanla diske yazilacak)
5. File offset pointer'i guncelle

4.5 Seek

ONEMLI NOKTA

Seek operasyonu yalnızca bellekteki (kernel'daki) “mevcut pozisyon” pointer'ini degistirir. Disk kolunu fiziksel olarak hareket ettirmez. Çok gorevli (multitasking) bir sistemde disk kolunu bu asamada hareket ettirmek anlamsizdir; gercek I/O ancak read/write sirasinda yapilir.

4.6 Dosya Acma ve Kapama Mekanizmasi – Detayli Sekil



5 Dizinler (Directories)

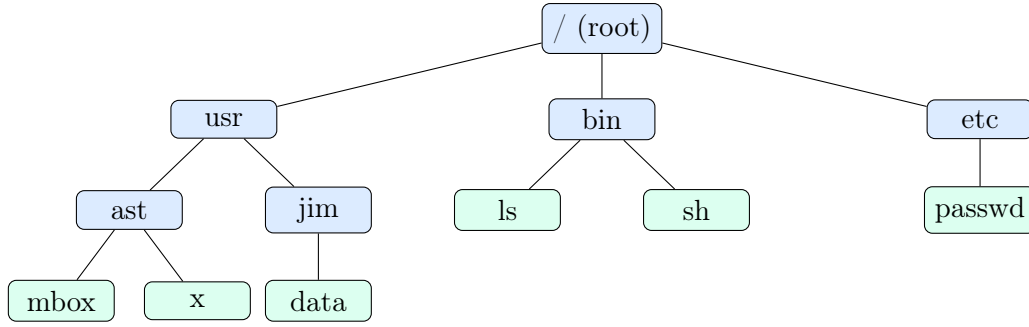
5.1 Dizin Yapilari

5.1.1 Tek Seviyeli Dizin

Tum dosyalar tek bir dizinde bulunur. Buyuk sistemlerde kullanisli degildir: farkli kullanicilarin ayni adla dosyasi olabilir, dosya sayisi artinca arama yavaslar.

5.1.2 Hiyerarsik Dizin

Modern sistemlerin kullandigi yaklasim. Dizinler, alt dizinlere sahip olabilir; alt dizinlerin de kendi alt dizinleri olabilir – bu sonsuza dek devam edebilir.



5.2 Mutlak ve Goreli Yollar

Mutlak yol (absolute path): Kok dizinden baslar. Ornek: `/usr/ast/mbox`

Goreli yol (relative path): Calisma dizininden (working directory) baslar. Eger calisma dizini `/usr/jim` ise, `../ast/x` dosyasi `/usr/ast/x`'e karsilik gelir.

“Calisma dizini `/usr/jim` iken, `../ast/x` goreli yolunun mutlak yolu nedir?”

Cevap: `/usr/jim` konumundayken `..` bir ust dizine cikar: `/usr`. Oradan `ast` dizinine girip `x` dosyasina erisiriz. Sonuc: `/usr/ast/x`

5.3 Dizin Operasyonlari

Sistem Cagrisi	Islevi
Create	Yeni bos dizin olustur (. ve .. otomatik eklenir)
Delete	Bos dizini sil (sadece . ve .. kalinca)
Opendir	Dizini okumak icin ac
Closedir	Dizin okumayi kapat
Readdir	Sonraki dizin girisini oku
Rename	Dizini yeniden adlandir (kernel uzerinden yapilir – hard link ile ayni mantik)
Link	Bir dizin girisi olustur (hard link)
Unlink	Dizin girisini kaldır

5.4 Dizin Uygulamasi (Directory Implementation)

Dizinler iki sekilde implemente edilebilir:

(a) Sabit boyutlu girisler

ad (16 B) | izin | uid | gid | boyut | blk0 | ...

ad (16 B) | izin | uid | gid | boyut | blk0 | ...

Sabit uzunlukta ad -> Uzun adlar kesilebilir

(b) i-node referansli

ad (degisken) | i-node no → i-node

ad (degisken) | i-node no

Metadata i-node'da; dizin hafif

5.4.1 Uzun Dosya Adlari

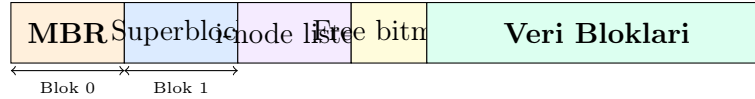
Degisken uzunluklu dosya adlari ile basa cikmak icin iki yaklasim:

- **In-line:** Her giriste degisken uzunluklu ad, bir uzunluk alanı ile birlikte tutulur. Basit ama silme ve ekleme islemleri karmasiklasar.
- **Heap:** Sabit boyutlu dizin girisileri bir pointer tutar; adlar ayri bir “heap” alanında tutulur. Daha esnek.

6 Disk Alan Yonetimi – Blok Tahsisi (Disk Space Management)

6.1 File System Genel Yerlesimi

Bir diskin tipik file system duzenine bakalim:



- **Boot record (MBR):** Diskin basinda, BIOS’un bulabildigi ilk bolgedir. Disk bolum tablosunu (partition table) ve boot kodunu icerir.
- **Superblock:** Dosya sisteminin kritik parametrelerini barindiran ozel bloktur. i-list’in nerede basladigi, blok boyutu, toplam blok sayisi, “clean shutdown” bayragi gibi bilgileri icerir.
- **i-list (i-node listesi):** i-node’ların disk üzerindeki dizisidir. i-node numarasi, bu diziye olan bir indistir.
- **Free block bitmap/liste:** Hangi blokların bos oldugunu takip eder.
- **Veri Bloklari:** Asil dosya icerigi buradadir.

6.2 Blok Boyutu Secimi

Blok boyutu kritik bir tasarim karardir. Çok kucuk bloklar: dosyalar çok parcanir, çok fazla i/o gerekir. Çok büyük bloklar: dahili parcalanma (internal fragmentation) olusur.

Ortalama blok transfer suresi:

$$T_{transfer} = T_{seek} + T_{rotation} + T_{data}$$

Buyuk bloklar: T_{data} baskindir, gecikme guncelleme hizi çok az etkiler.

Kucuk bloklar: Her blok icin ayri seek/rotation overhead’i dominate eder.

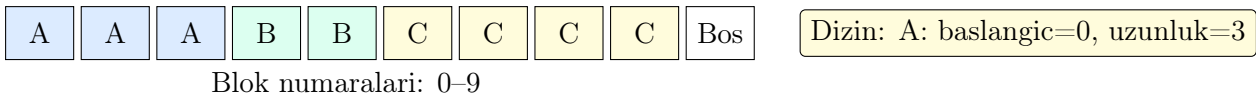
Blok boyutunun seciminde dikkate alınmasi gereken faktorler:

- **Veri hizi (data rate):** Buyuk bloklar, disk basina daha az seek demektir – daha yuksek bant genisligi.
- **Alan verimliliği:** Buyuk bloklar, kisa dosyalar icin bos alan birakmak anlamina gelir.
- **Tipik dosya boyutlari:** Eger cogu dosya 4KB'dan kucukse, 64KB blok boyutu israf olur.

6.3 Allocation Yontemleri – Karsilastirmali Inceleme

6.3.1 1. Contiguous Allocation (Surekli Tahsis)

Her dosya diskte bitisik (contiguous) blok dizisine yerlestirilir.



Avantajlar:

- Basit: Yalnizca baslangic blok + uzunluk bilgisi yeterlidir
- Hem sirasal hem de rastgele erisim icin cok hizlidir (disk kolu hareket etmez)

Dezavantajlar:

- **Dis parcalanma (external fragmentation):** Zaman icinde dosyalar silinip yenileri eklendikce kucuk bosluklar olusur; yeni buyuk dosya sigdirilamamaz
- Dosya buyudugunde tasinmak zorunda kalinir

“Contiguous allocation’da son blokta bos yer kalmasi – internal mi external mi fragmentation?”

Cevap: Bu **internal fragmentation**’dir. Tahsis edilen alan (blok) kullanilamayan bolumleri icerir ama o alan o dosyaya “atanmis” haldedir. Bir onceki bolumde calistigimiz segmentation ile analogi: bir segment icindeki kullanilmayan alan = internal fragmentation.

Varsayimlar: seek = 5 ms, rotational delay = 4 ms, transfer = 8 MB/s, ortalama dosya = 8 KB.

Bir dosyayı okuyup yeni konuma yazmak için:

$$T_{read} = T_{seek} + T_{rot} + \frac{8 \text{ KB}}{8 \text{ MB/s}} = 5 + 4 + 1 = 10 \text{ ms}$$

$$T_{write} = T_{seek} + T_{rot} + \frac{8 \text{ KB}}{8 \text{ MB/s}} = 5 + 4 + 1 = 10 \text{ ms}$$

Tek dosya için toplam: $10 + 10 = 20 \text{ ms}$.

16 GB diskin yarisi (**8 GB**) compaction edilecek. Ortalama dosya 8 KB:

$$\text{Dosya sayısı} = \frac{8 \text{ GB}}{8 \text{ KB}} = \frac{8 \times 2^{30}}{8 \times 2^{10}} = 2^{20} = 1,048,576 \text{ dosya}$$

$$\text{Toplam süre} = 1,048,576 \times 20 \text{ ms} \approx 20,971 \text{ s} \approx 349 \text{ dakika} \approx 5.8 \text{ saat}$$

Bu, contiguous allocation'ın neden pratik olmadığını açıkça göstermektedir.

6.3.2 2. Linked List Allocation (Bagli Liste Tahsisi)

Her dosya bir bagli liste olarak saklanır. Her blok, bir sonraki blogun adresini icerir. Dizin yalnızca ilk blogun adresini bilmek zorundadır.



Avantajlar:

- Alan verimli: Dis parçalanma yoktur
- Dosyaya ekleme (append) kolaydır

Dezavantajlar:

- **Yavaş rastgele erişim:** n'inci blogun adresine ulaşmak için n-1 blok okunmak zorundadır
- Her blokta bir pointer yer kaplar – veri alanı azalır
- Pointer bozulursa dosya kaybolur

6.3.3 3. File Allocation Table (FAT)

Bagli liste mantigi, ancak pointer'lar blokların içinde değil, bellekte tutulan bir tabloda (FAT) saklanır.

FAT Tablosu (RAM'de)

0	1
1	2
2	3
3	EOF
4	6
5	7
6	EOF
7	EOF

Dosya A: bloklar
0,1,2,3. Dizin: ilk
blok = 0

Dosya B: bloklar
4,6,7. Dizin: ilk
blok = 4

200 GB disk, 1 KB blok boyutu:

$$\text{Blok sayısı} = \frac{200 \times 2^{30}}{2^{10}} = 200 \times 2^{20} \approx 209,715,200 \text{ blok}$$

Her FAT girişi 4 byte ise:

$$\text{FAT bellek} = 209,715,200 \times 4 \text{ B} \approx 800 \text{ MB}$$

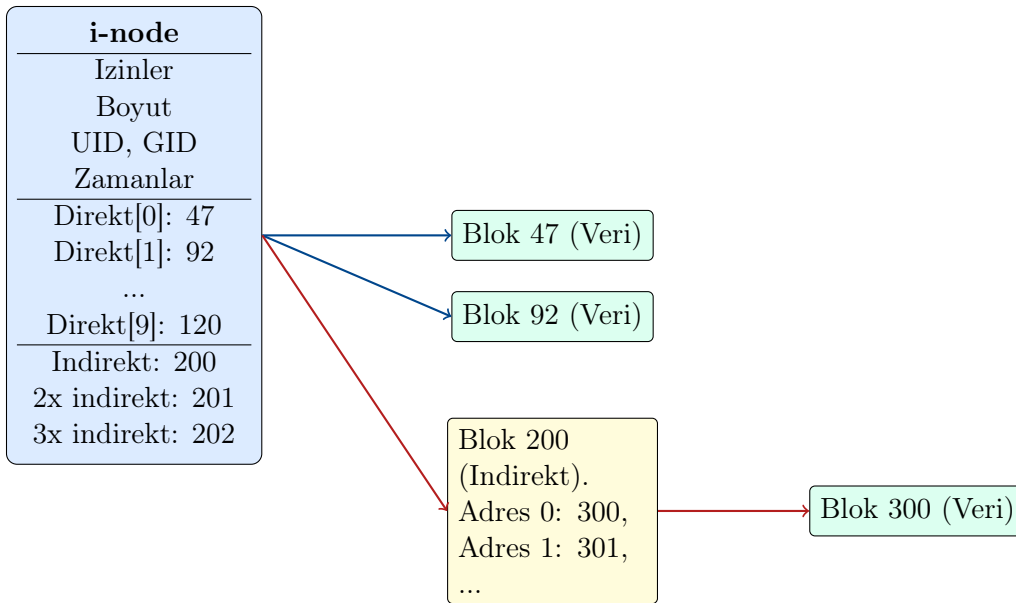
Bu FAT'ın tümü RAM'e sigdırılmak zorundadır – büyük diskler için büyük RAM talebi!

ONEMLI NOKTA – FAT Avantajı

FAT'ın en büyük avantajı: Disk bloklarını okumadan zinciri takip edebilirsiniz (hepsi RAM'de). Dezavantaj: Büyük diskler için FAT çok büyük olabilir; hala zinciri takip etmek $O(n)$ sürer.

6.3.4 4. Indexed Allocation – i-node

Bir dosyaya ait tüm blok adresleri **i-node** adlı özel bir yapıda bir arada tutulur. i-node'un belleğe alınması için dosya açılmış olmalıdır.



6.3.5 i-node ile Maksimum Dosya Boyutu

Varsayımlar: blok boyutu = B byte, pointer boyutu = P byte, her pointer başka bir bloğun adresini tutar.

- **Direkt bloklar** (d adet): $d \times B$ byte

- 1. seviye indirekt: $(B/P) \times B$ byte
- 2. seviye indirekt: $(B/P)^2 \times B$ byte
- 3. seviye indirekt: $(B/P)^3 \times B$ byte

$$\text{Maks. boyut} = d \cdot B + \frac{B}{P} \cdot B + \left(\frac{B}{P}\right)^2 \cdot B + \left(\frac{B}{P}\right)^3 \cdot B$$

“i-node 10 direkt adres, 4 byte’lik her adres, tum disk bloklari 1024 KB ise maksimum dosya boyutu?”

NOT: 1024 KB = 1024 × 1024 = 1,048,576 byte = 1 MB (blok boyutu buyuk!)

Her indirekt blogunda kac adres sigmaz? $\frac{1024 \times 1024}{4} = 262,144$ adres

Direkt : $10 \times 1,048,576 \approx 10$ MB

1x indirekt : $262,144 \times 1,048,576 \approx 256$ GB

Toplamda: ≈ 256 GB (1x indirekt blogu zaten cok buyuk dosyaya izin veriyor)

Kitabin kastettiği soruda “1024 KB” yerine muhtemelen “1024 byte = 1 KB” anlaminda.

O durumda: Her ind. blokta: $1024/4 = 256$ adres. 1x indirekt: $256 \times 1024 = 262,144$ byte Direkt: $10 \times 1024 = 10,240$ byte 2x indirekt: $256^2 \times 1024 \approx 64$ MB 3x indirekt: $256^3 \times 1024 \approx 16$ GB

“Blok boyutu 4 KB, pointer boyutu 4 byte, 11 direkt + 1 indirekt blok ile maksimum dosya boyutu?”

Blok boyutu: $4 \times 1024 = 4096$ byte

Indirekt blokta kac adres sigmaz? $\frac{4096}{4} = 1024$ adres

Direkt : $11 \times 4096 = 45,056$ byte = 44 KB

Indirekt : $1024 \times 4096 = 4,194,304$ byte = 4 MB

Toplam : $45,056 + 4,194,304 = 4,239,360$ byte ≈ 4.04 MB

6.3.6 Allocation Yontemlerinin Karsilastirmasi

Kriter	Contiguous	Linked	FAT	i-node
Sirasal erisim	Cok iyi	Kabul.	Iyi	Iyi
Rastgele erisim	Cok iyi	Cok kotu	Kotu	Iyi
Dis parcalanma	Var	Yok	Yok	Yok
Ic parcalanma	Son blokta	Yok	Yok	Son blokta
Metadata yeri	Dizin	Dizin	Dizin+FAT	i-node
Ekleme (append)	Zor	Kolay	Kolay	Kolay
Guvencilirlik	Dusuk	Orta	Orta	Yuksek
Kullanım yeri	CD-ROM	Eski sistemler	MS-DOS, USB	Unix, Linux

Soru 17: Ogrenci kayitlari, sabit boyutlu, rastgele erisim ve guncelleme: Cevap: **Indexed (i-node/FAT)** – Rastgele erisim hizlidir. Contiguous ise guncelleme sirasinda blok tasınabilir. Linked: Her kayda gitmek icin bastan baslamak gerekir.

Soru 18: Dosya boyutu 4KB ile 4MB arasında degisiyor: Cevap: **Indexed (i-node)**. Contiguous dinamik boyut degisimleri icin uygun degil; linked rastgele erisimi zorlarir; i-node hem kucuk hem buyuk dosyalari verimli yoneter.

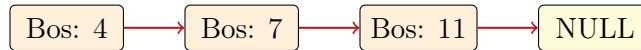
7 Bos Alan Yonetimi (Free Space Management)

7.1 Free List ve Bitmap Karsilastirmasi

Disk üzerindeki bos bloklar iki temel yontemle takip edilebilir:

7.1.1 Linked Free List

Bos bloklar, birbirlerine bagli bir liste olarak tutulur. İlk bos blok OS'in bildigi sabit bir konumda bulunur; her bos blok bir sonraki bos blogun adresini icerir.



7.1.2 Bitmap

Her blok icin 1 bit tahsis edilir. Bit 0 ise blok kullanilmiyor (bos); 1 ise dolu.

Disk 16 bloklu, blok 0 root dizin icin kullanilmis. Baslangic bitmap: 1 0000 0000 0000 000

(a) Dosya A yazildi, 6 blok kullanildi: 1111 1110 0000 0000

(b) Dosya A silindi (bloklar 1-6 serbest): 1000 0000 0000 0000

(c) Dosya C yazildi, 8 blok kullanildi: 1111 1111 1000 0000

(d) Dosya B silindi (bloklar 7-11 serbest): Not: Dosya B bloklari 7-11'i kullaniyordu (A silinmeden once bitiyor sonrasinda basliyor). Adim (a)'da A bloklari 1-6. Adim (b)'de A silindi. C bloklari tekrar 1-8. B'nin bloklari: onceki adim olan (a) sonrasinda B yazilsaydi bloklari 7-11 olurdu ama soru sirali. Soruyu yeniden okuyalim: A silindikten sonra C 8 blok kullanir -> 1-8. O zaman B, adim (a) den sonra yazilmis: B bloklari 7-11. D adiminda B silinince: 1111 1111 0000 0000 (sadece C'nin bloklari 1-8 dolu)

Disk: B toplam blok, F bos blok, adres boyutu D bit.

Bitmap boyutu: B bit = $B/8$ byte

Free list boyutu: $F \times D$ bit = $F \times D/8$ byte

Free list daha az yer kaplar, eger:

$$F \cdot D < B \implies F < \frac{B}{D}$$

$D = 16$ bit icin: $F < B/16$, yani diskin $1/16 = \%6.25$ 'inden az blok bos olmalı.

Pratikte diskler genellikle $> \%6.25$ bos olur, yani **bitmap genellikle daha verimlidir**.

“Bitmap veya free list tamamen kaybolursa ne olur?”

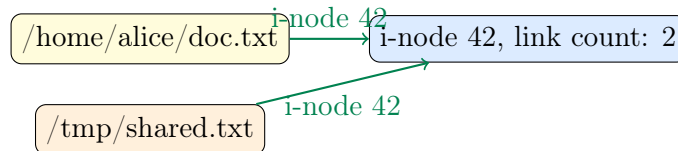
UNIX (i-node bazli): Bitmap kaybolursa, i-node'lar okunarak hangi blokların kullanımda olduğu yeniden hesaplanabilir. Her i-node'daki blok adresleri taranır, kalan bloklar bos kabul edilir. Bu işlemi **fsck** yapar.

FAT-16: FAT hem block allocation chain'i hem de free block bilgisini tutar. FAT tamamen kaybolursa, dosya yapisi kurtarılamaz (FAT'in icinde hem icerik zincirleri hem bos blok bilgisi birlikte). Veri kayıp olabilir.

8 Paylaşılan Dosyalar ve Linkler (Shared Files & Links)

8.1 Hard Link (Sabit Bağlantı)

Bir hard link, başka bir dizinde yeni bir dizin girişi oluşturur – ama bu giriş **aynı i-node numarasına** işaret eder. Dolayısıyla iki farklı "ad" aynı dosyayı gösterir.



- Link count = 2 (iki dizin girişi bu i-node'u gösteriyor)
- Bir dosya silindiğinde: link count 1 azalır. Yalnızca link count 0'a düştüğünde i-node ve bloklar gerçekten serbest bırakılır.

- Hard link **ayni file system** icinde olmak zorundadir (i-node numaralari partition'a ozgu)
- **Dizinlere** hard link **olusturulamaz** (dongu riski)

8.2 Symbolic Link (Sembolik Baglanti)

Sembolik link özel bir dosya tipidir; icerigi bir hedef dosyanin **yol adidir**. i-node gerektirmez (link dosyasinin kendi i-node'u vardir); hedef dosya farkli bir file system'da bile olabilir.

Soru 13: MacOS alias, sembolik link gibi davranir ama hedef dosyanin i-node numarasini ve boyutunu da saklar. Hedef dosya tasinirsa, alias hala calisir (i-node numarasıyla bulmaya calisir).

Avantaj: Dosya tasininca alias hala calisir; sembolik link kirilir.

Dezavantaj: Hedef dosya tasinip baska bir dosya eskihedef adresine yazilirsa, alias eski dosyaya devam eder – beklenmeyebilir.

Soru 14: MacOS 10.2+ te, hedef tasinip yerine yenisi konulursa, alias yeni dosyaya baglanir. Bu, yukardaki dezavantaji giderir ama avantajı azaltir (tasininca kirilebilir gibi davranabilir).

Kriter	Hard Link	Symbolic Link
i-node tahsisi	Yeni i-node yok; ayni i-node kullanilir	Yeni i-node olusturulur
Farkli partition	Hayir	Evet
Hedef silinirse	Dosya hala erisebilir (link count > 0)	Kopuk link (dangling link)
Dizine link	Hayir (dongu riski)	Evet (ama dikkatli olmalı)
Boyut	0 (ekstra depolama yok)	Yol adinin uzunlugu kadar

“Hard link ve soft link’in i-node tahsisindeki farki?”

Hard link: Iki farkli dizin girisi **ayni i-node numarasina** isaret eder. Ekstra i-node tahsisi yapilmaz. i-node'daki link count arttirilir.

Symbolic link: Sembolik link icin **yeni bir i-node olusturulur**. Bu i-node'un veri blogu, hedef dosyanin yolunu (path string'ini) icerir. Hedef dosyanin i-node'u etkilenmez.

9 Journaling File Systems

9.1 Cokme Sonrasi Tutarlilik Sorunu

Unix'te bir dosya silinirken uc adim atilir:

1. Dizin girisindenDosya kaldir

2. i-node'u bos i-node havuzuna don
3. Tum disk bloklarini bos blok havuzuna don

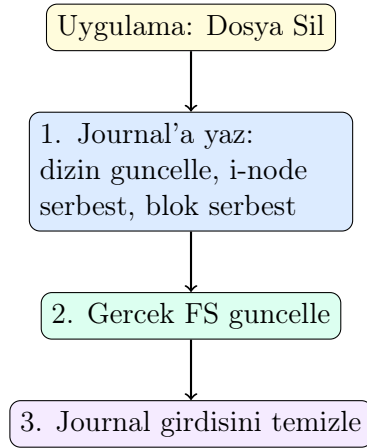
Bu adimlar arasindan guc kesilirse? Ornegin, adim 1 tamam ama 2 henuz gerceklesmemisse: i-node bir kimseye ait degil ama bos da sayilmiyor – **kaynak sızıntısı (resource leak)**.

ONEMLI NOKTA – Cokme Sonrasi Prensip

Yazma siralama kurali: Disk daima guvenli bir durumda olmalidir. Birden fazla yapi ayni anda guncellenmeden once, yazmalarin sirasi oyle planlanmalidir ki, guc kesilse bile disk tutarli bir durumda biraksin.

9.2 Journaling (Log-Structured / Journaling File System)

Verileri dogrudan guncelleme yerine, degisiklikler once bir **journal** (gunluk/log) alanina atomik sekilde yazilir. Sistem basariya ulasirsa, degisiklikler gercek konuma **commit** edilir.



*Guc kesintisi
burada olursa:
Boot'ta journal
replay edilir –
islem tamamlanir
ya da geri alinir*

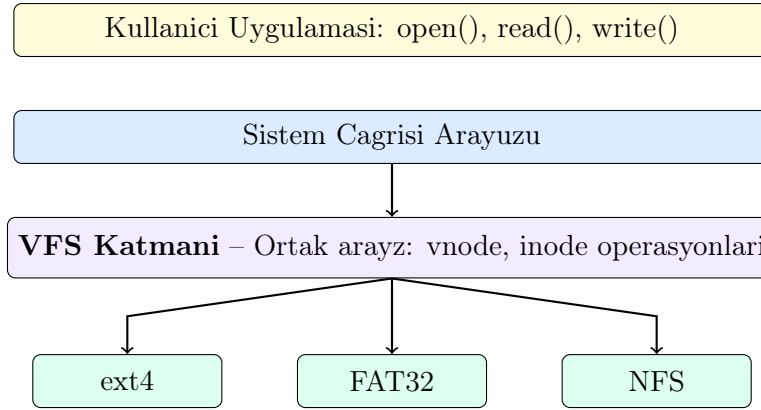
Modern diskler dahili tampon (buffer) kullanir (2-16 MB). Disk, yazmalari sirada degisin yeniden duzenliyebilir (seek optimizasyonu icin). Guç kesildiginde tampondaki veri kaybolur. Bu, journaling'in islevini zorlastirir. Cozum: Yazma sirasini zorla (flush/sync komutlari) ve bataryali write cache kullan.

10 Virtual File System (VFS)

10.1 VFS Nedir ve Neden Gereklidir?

Modern isletim sistemleri birden fazla dosya sistemi turunu destekler: ext4, FAT32, NTFS, NFS, procfs, tmpfs, vb. Kullanici programlari hangi file system'in kullanildigini bilmeden calisabilmelidir.

VFS (Virtual File System), uygulama katmani ile somut (concrete) file system implementasyonlari arasinda bir **soyutlama katmani** olusturur.



VFS, her somut file system'in implement etmesi gereken bir operasyon seti tanımlar (C'de fonksiyon pointer'ları ile; esasen nesne yönelimli polimorfizmin C implementasyonudur).

ONEMLI NOKTA – VFS ve Abstraction

“/home” ext4 üzerinde, “/cdrom” ISO 9660 üzerinde, “/net/server” NFS üzerinde olabilir. Kullanici `cat /net/server/dosya.txt` yazdığında, VFS hangi somut file system'in kullanıldığını seçip ilgili driver'i çağırır. Kullanici kodu değişmez.

11 File System Tutarlılığı ve Tamiri (Consistency & fsck)

11.1 Blok Tutarlılığı

`fsck` (Unix) veya `scandisk/chkdsk` (Windows), sistem çalışıyor sayılırken yüklenme sırasında çalışır ve file system tutarlılığını kontrol eder.

Blok tutarlılığı için iki sayac tablosu tutulur:

- **1. Tablo:** Her blok kaç dosyada geçiyor (i-node'lardan taranarak)
- **2. Tablo:** Her blok free list/bitmap'te kaç kez geçiyor

Durum	Tablo1 Tablo2	Cozum
Tutarlı	1 0 veya 0 1	İşlem gerekmez
Kayıp blok	0 0	Free list'e ekle
Serbest listede duplikat	0 2	Free list'ten birini sil
Dolu blok duplikatı	2 0	Blok içeriğini kopyala; her iki dosyaya birer kopya ver; kullanıcıya bildir

“Bir blok iki farklı dosyaya (`/home/hjb/dadjokes.txt` ve `/etc/motd`) tahsis edilmiş. `fsck` bloğu kopyalayıp her iki dosyaya ayrı kopyalar atıyor.”

(i) **Hangi durumda her iki dosyanın verisi doğru kalabilir?** Eğer iki dosyadan biri bloğu *hic kullanmadıysa* (dosya boyutu o bloğu kapsamı dışı)

inda) veya blok iceriginin iki dosyada da gecerli olabilecegi bir durumda – ornegin iki dosyada da bos alan veya bire bir ayni icerik varsa.

(ii) **Kullanici nasil arastirir?** `od -c`, `xxd`, veya `strings` komutu ile iki dosyayi da inceleyip iceriklerin anlamlı olup olmadigini kontrol edebilir.

(iii) **Veri kurtarma?** Yedeklerden (backup/snapshot), RAID'den, ya da disk forensics araclarıyla (TestDisk, PhotoRec) kurtarma denenebilir.

12 File System Performansi

Diske erisim bellek erisiminden **1000 kat** daha yavas. Performans iyilestirme icin uc temel teknik kullanilir:

12.1 1. Caching (Tampon Onbellekleme)

Disk bloklari bellekte cache'lenir. Bir blok okunacaksa once cache kontrol edilir. Cache'de varsa (**hit**): disk erisimine gerek yok. Yoksa (**miss**): diskten okunur, cache'e eklenir.

$$T_{avg} = h \cdot T_{cache} + (1 - h) \cdot T_{disk}$$

Ornegin: $T_{cache} = 1$ ms, $T_{disk} = 40$ ms:

$$T_{avg} = h \cdot 1 + (1 - h) \cdot 40 = 40 - 39h \text{ ms}$$

$h = 0.9$ icin: $T_{avg} = 40 - 39 \times 0.9 = 40 - 35.1 = 4.9$ ms

Cache hit orani ne kadar yuksek olursa performans o kadar iyi.

Cache yonetiminde LRU kullanilir, ancak i-node bloklari gibi ozel bloklar icin **degistirilmis LRU** kullanilir. Iki faktor dikkate alinir:

- O blok yakin zamanda tekrar gerekecek mi?
- O blok file system tutarlıligi icin kritik mi?

“Harici USB disk icin write-through cache mi yoksa block cache mi daha uygun?”

Write-through cache: Her yazma aninda hem cache hem de diske yazilir. Yavas ama guvenli.

Block cache (write-back): Yalnizca cache'e yazilir; blok daha sonra tembelce (lazily) diske aktarilir.

Cevap: USB disk icin **write-through** daha uygun. Kullanici USB diski cikardigi anda, cache'de diske yazilmamis dirty bloklar varsa veri kaybolur. Write-through bu riski ortadan kaldirir. (Not: Insan davranisini olarak USB'yi “Safely Remove Hardware” yapmadan cekerek, block cache ile blok kaybi olabilir – bu nedenle OS'lar USB aygitlar icin genellikle write-through kullanir.)

12.2 2. Block Read-Ahead (Blok Onokuma)

Dosyaya sirasal erisim tespit edilirse, bir sonraki blok henuz istek gelmeden arka planda okunur. Boyece, kullanıcı o blogu istediginde muhtemelen cache’de hazir olacak.

“Ogrenci kayitlari: ID girilir, kayit okunur/guncellenir/yazilir. Bu uygulama icin read-ahead yararlı midir?”

Cevap: Hayir. Read-ahead, *sirasal* erisim paterninden fayda saglar. Bu uygulama, her seferinde farkli bir ogrencinin kaydina *rastgele* erisir. Sonraki blok tahmin edilemez; onceden okunan blok buyuk ihtimalle kullanilmaz (wasted I/O). Hatta onbellek kirletilir (cache pollution).

12.3 3. Disk Kolu Hareketini Azaltma

i-node’lar genellikle diskin basinda tutulur. Ancak buyuk disklerde, i-node’dan veri bloguna olan seek uzun olabilir. Cozum:

Cylinder group: Disk, her biri kendi i-list, free list ve veri alanina sahip cylinder group’lara bolunur. Bir dosyanin i-node’u ve veri bloklari ayni cylinder group’ta tutulmaya calisilir – seek suresi azalir.

“Her Unix dosyasinin ilk kısmi i-node ile ayni disk blogunda tutulursa ne iyi olur?”

Bir dosya acildiginda i-node okunur. Eger dosyanin baslangic verisi de ayni blokta ise, ilk veri okumasi icin ikinci bir disk erisimine gerek kalmaz. Kucuk dosyalarda (tek blok sigan) dosyanin tamamı tek seferde okunur. Orta ve buyuk dosyalarda ilk blok hizlica gelir.

13 File System Yedeklemesi (Backup)

13.1 Yedekleme Stratejileri

1. **Felaket kurtarma:** Donanim ariza, yangin, sel gibi durumlarda veri kurtarma.
2. **Aptallik kurtarma:** Kullanicinin yanlilikla sildiginde ya da uzerine yazdiginda veri kurtarma.

13.1.1 Fiziksel Dump (Physical Dump)

- Blok 0’dan baslayip diskin tamamini tape’e kopyalar
- Guvenilir ve hizlidir

- Israf: bos bloklar, kotu bloklar da kopyalanir
- Artimsal (incremental) dump yapilamaz
- Yalnizca ayni geometrideki diske geri yuklenebilir

13.1.2 Mantiksal Dump (Logical Dump)

- Bir veya birden fazla dizinden baslayip dosyalari ozlemeli (recursively) dump eder
- Artimsal dump mumkun: Yalnizca son dump'tan bu yana degisen dosyalar dump edilir
- Unix'te `dump/restore`, dump **seviyeleri** kullanir: 0 = tam, 1 = 0'dan bu yana degisen, vb.

13.1.3 Mantiksal Dump Algoritmasi (4 Faz)

1. **Faz 1:** Tum degistirilmis dosyalari ve (degistirilmis olsun olmasin) her dizini isaretle.
2. **Faz 2:** Icerisinde degistirilmis dosya bulunmayan dizinlerin isaretini kaldir.
3. **Faz 3:** i-node numaralari sirasinda taranarak isaretli dizinleri dump et.
4. **Faz 4:** Isaretli dosyalari dump et.

Soru 27: Oliver Owl, yedekleme yapilan sistemde tez yaziyor. Sorun mu?

Evet – Baslangic tam dump alinirken tez dosyasi degistiriliyor. Dump baslangicinda alinan dosya versiyonu ile dump bitis sonraki kopya senkronize olmayabilir; tape'e yanlis veya yarim dosya kopyas yazilabilir.

Soru 28: Unix'te archive bit (Windows'ta var) olmadigi icin nasil artimsal dump yapilir? Cevap: **mtime** (degistirme zamanı) kullanilir. Son dump'in tarihi bilinir; o tarihten bu yana mtime degismis her dosya dump edilir.

Soru 29: Dosya 21 son dump'tan bu yana degistirilmemis ise bitmap'ler nasil farkli olur? Faz 1'de dosya 21 isaretlenmez (degistirilmemis). Faz 3-4'te de dahil edilmez. Dolayisiyla dosya 21'e ait bit, tum dort bitmap'te 0 olur.

13.2 Kurtarma Algoritmasi

1. Bos bir file system olustur
2. En son tam (full) dump'u geri yukle: Once dizinleri, sonra dosyalari
3. Sira ile her artimsal dump'u geri yukle

Sorunlar:

- Bos bloklar: Kurtarma tamamlandiktan sonra yeniden olustur
- Hard linkler: Iki dizinde bulunan bir dosya yalnizca bir kez geri yuklenmeli; birden fazla yuklenirse duplikat olusur

14 Dosya Sistemi Ornekleri

14.1 ISO 9660 – CD-ROM Dosya Sistemi

- CD-ROM'lar için tasarlanmış, salt okunur (read-only)
- İlk 16 blok boş (OS'a veya başka kullanıcıya ayrılmış)
- **Primary Volume Descriptor:** CD-ROM hakkında genel bilgi (tittle, olusturucu, blok boyutu vb.)
- Dizin girişleri sabit uzunluklu; dosya adları 8+3 karakter (kısak form)

14.1.1 Rock Ridge Uzantıları

POSIX uyumluluğu için eklenmiştir:

- **PX:** POSIX izinleri
- **SL:** Sembolik link
- **NM:** Alternatif (uzun) dosya adı
- **TF:** Zaman damgaları

14.1.2 Joliet Uzantıları

Windows uyumluluğu için Microsoft tarafından eklenmiştir:

- Uzun dosya adları (64 karakter)
- Unicode karakter seti
- 8 seviyeden derin dizin iç içe geçmesi
- Uzantılı dizin adları

14.2 MS-DOS / FAT Dosya Sistemi

- Root dizin sınırlı boyutta (FAT-12/16: sabit girişi sayısında)
- Dosya adları: 8+3 format (uzun adlar sonraki versiyonlarda eklendi)
- Metadata dizin girişinde tutulur (Unix'in aksine – i-node yok!)
- Dizin girişi FAT tablosu girişini işaret eder

10 veri bloku (14-23). Dosya f1'in ilk blou 22, f2'nin ilk blou 16. FAT: (14,18),(15,17),(16,23),(17,21),(18,20),(19,15),(20,-1),(21,-1),(22,19),(23,14)
f1 zinciri (22'den baslayip): 22 -> FAT[22] = 19 -> FAT[19] = 15 -> FAT[15] = 17 -> FAT[17] = 21 -> FAT[21] = -1 (son) f1 bloklari: **22, 19, 15, 17, 21**
f2 zinciri (16'dan baslayip): 16 -> FAT[16] = 23 -> FAT[23] = 14 -> FAT[14] = 18 -> FAT[18] = 20 -> FAT[20] = -1 (son) f2 bloklari: **16, 23, 14, 18, 20**

FAT-16: 64K = 65,536 giris. Her giris 16 bit (1 giris = 1 blok).

Soru 41: 32,768 giris olsaydi:

Blok boyutu = 32 KB (maksimum) $\implies$ Maks. partition = $32,768 \times 32 \text{ KB} = 1 \text{ GB}$

Ama standart FAT-16'da 65,536 giris ile 2 GB'a kadar partition desteklenir.

Soru 42: Bir dosya k giris kullaniyorsa:

Tum dosyalarin toplam uzunlugu $\leq 65536 \times \text{blok boyutu}$

Yani toplam dosya uzunlugu, FAT tablo boyutu ile sinirlidir.

14.2.1 Uzun Ad Destegi (Windows 98+)

Geriye donuk uyumlulugu korumak icin yapilan zekice bir hile:

1. Uzun adindan 8.3 uyumlu kısa bir versiyon uretilir
2. Uzun adi tutan sahte (invalid), ozel isaretli directory entry'ler gercek entryin onune eklenir
3. Eski DOS bu girdileri gecersiz sayip gorur
4. Checksum, kısa adli gercek dosyanin silinip yerine yeni bir tane konulmasini fark eder

14.3 UNIX V7 Dosya Sistemi

- Directory entry: Sabit boyutlu, 16 byte'lik ad + 2 byte i-node numarası (toplam 16+2=18 byte)
- i-node: 10 direkt + 1 tekli + 1 çift + 1 uclu indirekt adres
- Tum metadata i-node'da; dizin yalnızca ad-inode eslemesini tutar

14.3.1 Dosya Bulma: /usr/ast/mbox

Root i-node bellekte. Diger hic bir sey bellekte degil. Tum dizinler tek bloga sigiyor.

1. **Root dizin bloku oku** (1 disk erisimi): “usr” arani -> i-node 6 bulundu
2. **i-node 6 oku** (1 disk erisimi): “usr” dizininin i-node’u
3. **“usr” dizin bloku oku** (1 disk erisimi): “ast” araniyor -> i-node 26 bulundu
4. **i-node 26 oku** (1 disk erisimi): “ast” dizininin i-node’u
5. **“ast” dizin bloku oku** (1 disk erisimi): “courses” araniyor -> i-node 60 bulundu
6. **i-node 60 oku** (1 disk erisimi)
7. **“courses” dizin bloku oku** (1 disk erisimi): “os” araniyor
8. **i-node “os” oku** (1 disk erisimi)
9. **“os” dizin bloku oku** (1 disk erisimi): “handout.t” bulundu
10. **“handout.t” i-node oku** (1 disk erisimi)

Toplam: 10 disk erisimi (root i-node bellekte, her dizin icin i-node okuma + dizin blok okuma)

15 Unix Dosya Sistemi Islemleri – Derinlemesine Inceleme

15.1 . ve .. Dizin Girisileri

Her dizinde iki ozel giris vardir:

- . (nokta): Mevcut dizinin i-node numarasina isaret eder
- .. (cift nokta): Ust dizinin i-node numarasina isaret eder

Bunlar sadece birer dizin girisidir; cekirdek icin hicbir ozel anlami yoktur. Bir yol takibi yapilirken “.” mevcut dizinden tekrar baslamak anlamina gelir; “..” ust dizine gitmek anlamina gelir. Bu mekanizma, dizin yapisinin agac olmasi gerektigini gerektirmez – yol takibi agac yapisi varsaymaz.

15.2 Dosya Olusturma (Unix)

1. Dizin girisin bak: Varsa, acma gibi (ama truncate gerekebilir).
2. Yoksa: Dizine yazma gerekir (dizine yazma = normal dosyaya yazma gibi; dizin de bir dosyadir).

3. Bos i-node bul ve tahsis et. i-node numarası 0 olan girişler bos i-node sagilir; link count 0 olan i-node'lar bos i-node'dur.
4. Yeni dizin girisin yaz.

15.3 Dosya Silme (Unix) – Unlink Mekanizması

- Link count (i-node'daki) bir azaltılır.
- Eger link count hala > 0 : Dosyanın baska adlari var; i-node ve bloklar korunur.
- Eger link count = 0 **VE** referans count (in-memory, acik dosya sayisi) = 0: Bloklar serbest bırakılır.
- **Onemli:** Acik bir dosyayi unlink edebilirsiniz! Dosya gercekten silinmez; son `close()` yapildiginda silinir. Bu, isletim sistemlerindeki klasik bir ozelliktir: Surec acik oldugu surece dosyaya erisebilir.

“Acik bir dosya unlink edilirse ve sistem cikilirse ne olur?” – Dosya icin i-node'daki link count 0, reference count 0'a duşmedi (cikildi). Disk üzerinde o i-node ve bloklar hala tahsisli görünür. Sonraki `fsck` calistirmasinba bu kayip bloklar tespit edilir ve serbest bırakılır.

15.4 Dizin Olusturma ve Silme

Dizin olusturma:

1. Normal dosya olusturma gibi.
2. Kernel, “.” ve “..” girişleri *otomatik* olarak yazar (kullanici bunu yapamaz; `write()` ile dizine doğrudan yazma yasak).
3. Ust dizinin link count'u arttirilir (cunku “..” ust dizine isaret eder).

Dizin silme:

1. Dizin yalnızca “.” ve “..” iceriyorsa silinebilir.
2. Normal dosya silme gibi.
3. Ust dizinin link count'u azaltılır.

16 i-Node – Kapsamli Inceleme

16.1 i-node Nedir?

i-node Tanimi

i-node (index node): Unix dosya sisteminde her dosya icin tutulan, o dosyaya ait tum metadata'yi ve disk blok adreslerini iceren sabit boyutlu bir veri yapisidir. Dosyanin adi i-node'da **degildir**; ad yalnızca dizin girisin de bulunur.

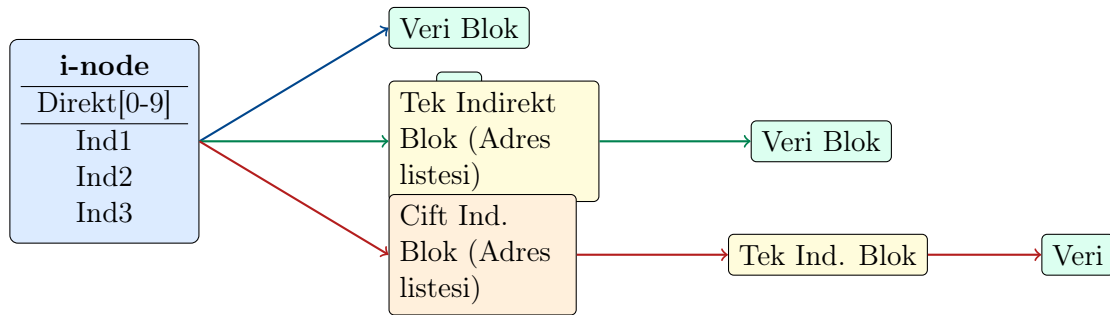
i-node'lar bir **i-list** (i-node listesi) icinde diskte saklanir. i-node numarasi bu listenin indisidir.

16.2 i-node'da Ne Var?

- **stat** yapisinin tum alanlari (mod, uid, gid, boyut, zamanlar, vb.)
- Kullanicinin degistirebilecegi bazi ek bayraklar (flags)
- Disk blok bilgileri: Direkt blok adresleri + tek/cift/uclu indirekt blok adresleri

16.3 i-node ile Buyuk Dosya Destegi

“Her problemin yazilimda bir cozumu, bir ek soyutlama katmani eklemekten gecer.” – David Wheeler



“Kisa dosyalar icin veri dogrudan i-node’a saklanabilir mi?”

Unix V7 tipik i-node yapı:

- 10 direkt adres $\times$ 4 byte = 40 byte
- 3 indirekt adres $\times$ 4 byte = 12 byte
- Toplam adres alanı = 52 byte

Eger bu alanlar veri olarak kullanilsaydi, 52 byte’a kadar olan dosyalar tek disk erisiminde okunabilirdi. Gercekte, modern ext4 gibi dosya sistemleri bu optimizasyonu **inline data** ozelligi olarak sunar – buyuk avantaj kucuk dosyalar icin!

“Carolyn: Dosya acildiginda her zaman yeni bir i-node kopyasi bellege al. Elinor: Onun yerine mevcut tabloyu ara. Kim hakli?”

Elinor hakli. Eger ayni dosya birden fazla kez acilrsa (farkli surecler veya ayni surec tarafından), her seferinde yeni bir kopya almak:

- Bellek israf eder
- Daha onemli: I-node üzerindeki degisiklikler (boyut, mtime gibi) diger kopyalara yansimaz – **tutarsizlik!**

i-node tablosu arama yaparak, ayni i-node’un tek bir in-memory kopyasi tutulur; degisiklikler herkese yansir.

17 Disk Kotasi (Disk Quota)

Kullanicilarin disk kullanimini sinirlamak icin **kota tablosu** tutulur. Her kullanıcı için:

- **Yumusak sinir (soft limit):** Asimda uyarı verilir; sinirli sure asimda kalinabilir.
- **Sert sinir (hard limit):** Hicbir kosulda asilamaz.

Her dosya acildiginda OS, o dosyanin sahibini kotasiyla karsilastirir. Dosya yazmalarında kota ayni sekilde takip edilir.

18 Kapsamli Soru Cozumleri

18.1 Teorik ve Kavramsal Sorular

Soru 1 – Dosya Turu Belirleme

Windows’ta bir dosya çift tiklandiginda hangi program calisiyor? **İki yontem:**

1. **Dosya uzantisi:** Registry’deki uzanti-program eslemesi: `.pdf` -> Adobe Reader, `.docx` -> Word.
2. **Magic number:** Dosyanin iceriginin basindaki baytlara bakilir. Ornegin, PDF: `%PDF`, EXE: `MZ`, JPEG: `FF D8 FF`.

Windows birincil olarak uzantiyi kullanir; UNIX ise `file` komutu magic number’i kullanir.

Soru 5 – Rastgele Erisimli Dosyalarda Rewind

“Rastgele erisimi destekleyen dosya sistemleri rewind’e ihtiyac duyar mi?”

Cevap: Teknik olarak gerekmez – `seek(0)` ile baslangica gitmek ayni islevi gorur. Ancak bazi uygulamalar rewind’i kullanmak isteyebilir ve kolaylik acisindan varligi mantiklidir. Gereklik degil, kolaylik.

Soru 6 – Rename vs Kopyala+Sil

“`rename()` sistem cagrisi mi yoksa kopyala+sil mi?”

Onemli fark: `rename()` *atomik* bir islemdir (tek islemden tamamlanir). Kopyala+sil iki adimlidir:

1. Kopyalama sirasinda cokme: Her iki kopya da yarimasinda; biri eksik
2. Kopyala tamam ama sil gerceklesmeden cokme: Iki kopya var, biri eski

Ayrıca `rename()` ile dosya icerigi diskte tasinmaz; yalnızca izin girisileri guncellenir – çok daha hizlidir.

Soru 7 – Tek Dizinli Sistemde Hiyerarsik Simulasyon

“Yalnızca tek izin var, rastgele uzunlukta ad var. Hiyerarsik sistem simule edilebilir mi?”

Evet! Dosya adlarında ‘/’ karakteri kullanılarak hiyerarsik yap simule edilebilir: `usr/ast/mbox`, `usr/jim/data`, vb.

Ancak OS bu yapıdan habersizdir; uygulama katmanı işlemleri kendi gerçekleştirmek zorundadır. Dizin silme (recursive) ve izin listeme gibi operasyonlar karmaşıklaşır.

Soru 8 – Seek Olmadan Rastgele Erisim

“Seek sistem çağrısı olmadan rastgele erişim nasıl sağlanır?”

Alternatif: `read/write` sistem çağrılarında bir *byte offset* parametresi ekle: `read(fd, buffer, n, offset)` – offset’ten başlayarak n byte oku.

Bu yaklaşım: POSIX’te `pread()` ve `pwrite()` olarak mevcuttur (seek + read/write yerine atomik olarak pozisyon ve okuma/yazma birlikte yapılır).

Soru 15 – Contiguous Allocation Uygun Cihaz

“Hangi modern cihaz contiguous allocation için uygundur?”

Cevap: DVD veya Blu-ray diske kaydeden kameralar. Bir video dosyası başladıktan sonra hep sırasal yazılır; dosya boyutu önceden bilinir (sabit bit hızlı kayıt). Parçalanma sorun olmaz çünkü dosyalar kaydedildikten sonra silinmez. Ayrıca CD-ROM/DVD yazma sırasında contiguous’tur.

Soru 21 – Hard vs Symbolic Link Avantajları

Hard link avantajı: Hedef dosya başka bir yola tasınsa veya adlandırılrsa bile, hard link çalışmaya devam eder (çünkü aynı i-node numarasına işaret eder – ada değil).

Symbolic link avantajı: Farklı file system’lar arasında ya da farklı partition’larda çalışan dosyalara link verebilir. Dizinlere de link verilebilir.

Soru 23 – FAT Blok Kac Adres?

“4 TB disk, 8 KB blok, free-list yöntemi. Bir blokta kaç adres var?”

Blok boyutu: $8 \text{ KB} = 8 \times 1024 = 8192 \text{ byte}$.

4 TB disk, 8 KB blok:

$$\text{Blok sayısı} = \frac{4 \times 2^{40}}{8 \times 2^{10}} = \frac{2^{42}}{2^{13}} = 2^{29} \approx 537 \text{ milyon blok}$$

Bu kadar bloğu adreslemek için gereken minimum bit sayısı: $\lceil \log_2(2^{29}) \rceil = 29 \text{ bit}$, yani en az 4 byte (32-bit adres).

Bir blokta sigmasi gereken pointer sayisi:

$$\frac{8192 \text{ byte}}{4 \text{ byte/adres}} = 2048 \text{ adres}$$

Soru 31 – fsck: Her iki tabloda da deger 2

“Hem “dosyada” hem “free liste’de” sayaci 2 olabilir mi?”

Evet! Eger bir blok hem bir i-node’da iki kez listelenmis hem de free list’te iki kez geciyor ise, her iki sayac 2 olabilir. Ya da iki farkli dosya ayni bloku paylasiyor VE blok ayrica free list’te de var.

Duzeltme: Blogu free list’ten kaldir (dosyaya ait), sonra dosyadaki duplikat adresi de duzelt (bloku kopyala, ikinci kullanıcı için yeni blok kullan).

Soru 39 – SSD, Aşınma ve Sıkıştırma

“SSD’lerde sıkıştırma neden hücre aşınmasını azaltır?”

SSD’lerde her hücre sinirli sayıda yazılabilir. Sıkıştırma şunu sağlar:

- Daha az veri fiziksel olarak yazılır (aynı mantıksal veri daha az byte ile temsil edilir).
- Daha az yazma = daha az hücre aşınması.

Örneğin, 100 KB veri 70 KB’a sıkışır, 30 KB’lık yazma işlemi gerçekleşmez – bu uzun vadede anlamlı bir fark yaratır.

Soru 37 – Extended Attribute Yaklaşımı

“Dosya tipini extended attribute olarak saklama: uzantıya göre mi, içerik analizine (magic number) göre mi daha iyi/kötü?”

Uzantıya göre avantajı: Kullanıcı istediği uzantıyı seçer; esnektir.

Dezavantajı: Kullanıcı uzantıyı değiştirirse yanlış program açılır.

İçerik analizine göre avantajı: Gerçek dosya tipini yansıtır; uzantı yanlış bile olsa doğru program seçilebilir.

Dezavantajı: Her dosyayı açmadan önce analiz gerektirir.

Extended attribute avantajı: Metadata’da saklı; hızlı erişim. Kullanıcı veya uygulama tarafından bir kez belirlenir. Hem uzantı hem magic number’ın getirdiği sorunları önler – ama sadece değer doğru set edilmişse çalışır.

Dezavantajı: Dosya başka sisteme kopyalanırsa extended attribute kaybolabilir.

Soru 44 – i-node Dosyanın İlk Bloğunda

“i-node’u diskin başında değil, dosyanın ilk bloğunda saklamak?”

Avantajlar:

- Dosyanın verisi ve i-node’u yan yana – tek seek ile ikisi de okunabilir.
- Küçük dosyalar için büyük performans kazanımı.

Dezavantajlar:

- i-node'lar artık sabit konumda değil – her dosyayı bulmak için ne i-node ne de disk başından taramak zorunda kalınır.
- fsck gibi tutarlılık araçları i-node'ları bulmakta zorlanır.
- Hard link uygulamak güçleşir (birden fazla dizin girişi aynı i-node'a işaret etmeli ama i-node nerede?).

19 Kavram Sozlugu

Terim	Aciklama
Allocation	Dosya icin disk bloklarinin tahsis edilmesi
Archive bit	Windows'ta dosyanin yedeklenmesi gerekip gerekmedigini gosteren bit
Bitmap	Disk bloklarının dolu/bos durumunu bit dizisi ile gosteren yapı
Block	Diskte okunan/yazilan minimum birim (genellikle 512B-64KB)
Boot record/MBR	Diskin ilk sektoru; BIOS buradan baslar
Buffer cache	Disk bloklarini bellekte tutan on bellek katmani
Contiguous allocation	Dosyanin bitisik disk bloklarina yerlestirilmesi
Cylinder group	Diskin i-list, free list ve veri alanini iceren bolumu
Dangling link	Hedefi silinmis olan sembolik link
Directory	Ad-inode eslemelerini tutan dosya
dump	Dosya sisteminin yedekkopya olarak tape/diske yazilmasi
External fragmentation	Tahsis edilmemis ama kullanilamaz kucuk bosluklar
FAT	File Allocation Table; bagli liste zincirini RAM'de tutan tablo
File descriptor	Acik dosyayi temsil eden tam sayi; open() tarafından doner
fsck	Unix'te file system tutarlilik denetcisi
Hard link	Ayni i-node'a isaret eden baska bir dizin girişi
i-list	i-node'ların disk üzerindeki dizisi
i-node	Dosya metadata'si ve blok adreslerini tutan yapı
Indexed allocation	Blok adreslerini i-node'da toplayan yontem
Internal fragmentation	Tahsis edilen alamin kullanilmayan kısmi
Journaling	Degisikliklerin once log'a yazildigi file system yaklasimi
Link count	i-node'a isaret eden dizin girişi sayısı
Linked allocation	Dosyanin bagli liste seklinde disk bloklarina yayilmasi
Logical dump	Dizin agacini dikkate alan mantiksal yedekleme
Magic number	Dosyanin turunu belirlemek icin kullanilan ozel baslagic byte'leri
Metadata	Dosya hakkında veri (boyut, izin, zaman, blok adresleri)
Physical dump	Disk bloklarini dogrudan kopyalayan fiziksel yedekleme
Read-ahead	Sonraki blogu istekten once arka planda okuma
Superblock	File system parametrelerini tutan ozel blok
Symbolic link	Hedef dosyanin yolunu icerik olarak saklayan ozel dosya

Terim	Aciklama
VFS	Virtual File System; farkli file system'lari ortak arayuz altinda sunan katman
Write-back cache	Dirty bloklari geciktirilmis olarak diske yazan on bellek
Write-through cache	Her yazmayi aninda diske de yansitan on bellek

20 Sinav Cevap Kaliplari

“X allocation’in dezavantaji nedir?” soru kaliplari:

- Contiguous -> “dis parcalanma, dosya buyuyunce tasima gerekir”
- Linked -> “rastgele erisimde her blogu okumak gerekir, O(n) disk I/O”
- FAT -> “FAT tum bellege sigmali; buyuk diskler icin buyuk RAM”
- i-node -> “cok seviyeli indireksiyonda ekstra disk erisimleri”

“Neden i-node? Neden dizin girisin de degil?” sorusu: Hard link’i desteklemek icin: iki farkli dizin girisi ayni i-node’a isaret edebilir. Metadata dizin girisin de olsaydi, kopyalari senkronize tutmak zorunda kalinirdik.

Maksimum dosya boyutu hesabi:

$$\text{Maks} = d \cdot B + \frac{B}{P} \cdot B + \left(\frac{B}{P}\right)^2 \cdot B + \left(\frac{B}{P}\right)^3 \cdot B$$

fsck tutarlilik sorunu cevap sablonu: “Tki tabloda [deger] olmasi, [ne anlama geldigini] gosterir. Duzeltme: [hangi tablodan ne kaldırilir/eklenir].”

“Journaling neden guvenlidir?” cevap sablonu: “Degisiklikler once atomik olarak journal’a yazilir. Sistem cokse, boot’ta journal replay edilir; ya tamamlanir ya da geri alinir. Disk hic yarim halde kalmaz.”

21 Final Tekrari – 30 Kritik Madde

1. File system, dosya adlarini fiziksel ortamdaki bolgelere eslediginden (mapping), OS’in depolama kaynaklari icin soyutlamasidir.
2. Uzun vadeli depolama icin uc gereksinim: cok buyuk veri, kalicilik (non-volatile), cok surec erisimi.
3. Unix, dosyalari salt bir byte dizisi (byte sequence) olarak gorur; icerik anlami uygulamaya birakilir.
4. **Metadata** dosya hakkındaki veridir: boyut, izinler, sahip, blok adresleri, zaman damgalari.

5. Unix'te **atime** / **mtime** / **ctime**: erişim / içerik değişimi / metadata değişimi. *ctime oluşturma zamanı* değildir!
6. **i-node**: Unix'te dosyanın metadata ve blok adreslerini tutan yapı. Dizin yalnızca ad + i-node no saklar.
7. $\text{ad} \rightarrow \text{dizin girişi} \rightarrow \text{i-node no} \rightarrow \text{i-node} \rightarrow \text{blok adresleri} \rightarrow \text{veri}$
8. **open()** sistemi çağırışı: Dosyayı bir kez bulur, izni bir kez kontrol eder, file descriptor döner. Performans kritik.
9. Hard link: Aynı i-node'a başka bir dizin girişi. Link count artar. Farklı partition'a olamaz. Dizine olamaz.
10. Symbolic link: Kendi i-node'u var, içeriği hedef yoludur. Farklı partition'lara olabilir. Hedef silinirse dangling link.
11. **Contiguous allocation**: Basit, hızlı, ama disk parçalanma ve dögama sorunları.
12. **Linked allocation**: Disk parçalanma yok, append kolay, rastgele erişim çok yavaş.
13. **FAT**: Linked listin pointer'larını RAM'e tasir; disk erişimine gerek kalmaz ama FAT bellege sigmalı olmalı.
14. **i-node (indexed) allocation**: Rastgele erişim hızlı, disk parçalanma yok, tek/cift/üçlü indirekt ile büyük dosyaları destekler.
15. Maksimum dosya boyutu: $d \cdot B + (B/P) \cdot B + (B/P)^2 \cdot B + (B/P)^3 \cdot B$
16. **Bitmap**: Her blok için 1 bit. Genellikle free list'ten daha verimlidir (diskin 1/D'den fazlası boş ise).
17. fsck iki tablo tutar: her bloğun “dosyalarda kaç kez” ve “free list'te kaç kez” geçtiğini sayar.
18. **Journaling**: Değişiklikler önce journal'a atomik yazılır; commit sonra gerçek FS'ye. Çokme sonrası tutarlılık.
19. **VFS**: Farklı file system implementasyonları üzerinde ortak arayüz. Uygulama kodu değişmeden ext4/FAT/NFS kullanabilir.
20. Cache hit oranı h için ortalama erişim süresi: $T = h \cdot T_{\text{cache}} + (1 - h) \cdot T_{\text{disk}}$
21. **Read-ahead**: Sirasıl erişimde sonraki bloku henüz istenmeden arka planda oku. Rastgele erişimde israf.
22. **Cylinder group**: i-list + freelist + veri aynı bölümde. Dosya i-node ve blok yakın tutulur, seek azalır.
23. **Physical dump**: Tüm bloklar; boş, kötü bloklar da kopyalanır. Artımsal dump olamaz.
24. **Logical dump**: Dizin ağacından başlayarak değiştirilmiş dosyaları dump eder. 4 fazlı algoritma.

25. Unix'te artimsal dump: archive bit yok; **mtime** ile son dump tarihinden sonra degisenleri bul.
26. Unlink: Link count azaltilir. Sifira duserse bloklar serbest. Acik dosya unlink edilebilir: gercek silme son close()'ta olur.
27. FAT-16: 65,536 giris, metadata dizin girisinde, kisa 8.3 adlar. Uzun adlar sahte gecersiz girislerle desteklendi.
28. ISO 9660: CD-ROM icin, salt okunur. Rock Ridge (POSIX), Joliet (Unicode/uzun ad) uzantilari.
29. Seek: Yalnizca in-memory dosya pozisyon pointer'ini gunceller; disk kolu hareket etmez.
30. Dosya boyutu 4KB-4MB degisiyorsa: **indexed (i-node)** en uygun – dinamik boyut, hizli rastgele erisim, parcalanma yok.

Basarilar! – CSE 312 File Systems Bolumu